



Introduction to C++ Programming, Input/ Output and Operators



2.1 Introduction

- ▶ We now introduce C++ programming, which facilitates a disciplined approach to program development.
- ▶ Most of the C++ programs you'll study in this book process data and display results.

2.2 First Program in C++: Printing a Line of Text



- ▶ Simple program that prints a line of text (Fig. 2.1).



```
1 // Fig. 2.1: fig02_01.cpp
2 // Text-printing program.
3 #include <iostream> // allows program to output data to the screen
4
5 // function main begins program execution
6 int main()
7 {
8     std::cout << "Welcome to C++!\n"; // display message
9
10    return 0; // indicate that program ended successfully
11 } // end function main
```

Welcome to C++!

Fig. 2.1 | Text-printing program.



2.2 First Program in C++: Printing a Line of Text (cont.)

- ▶ `//` indicates that the remainder of each line is a **comment**.
 - You insert comments to document your programs and to help other people read and understand them.
 - Comments are ignored by the C++ compiler and do not cause any machine-language object code to be generated.
- ▶ A comment beginning with `//` is called a **single-line comment** because it terminates at the end of the current line.
- ▶ You also may use comments containing one or more lines enclosed in `/*` and `*/`.



2.2 First Program in C++: Printing a Line of Text (cont.)

- ▶ A **preprocessing directive** is a message to the C++ preprocessor.
- ▶ Lines that begin with **#** are processed by the preprocessor before the program is compiled.
- ▶ **#include <iostream>** notifies the preprocessor to include in the program the contents of the **input/output stream header file <iostream>**.
 - This header is a file containing information used by the compiler when compiling any program that outputs data to the screen or inputs data from the keyboard using C++-style stream input/output.

2.2 First Program in C++: Printing a Line of Text (cont.)



- ▶ You use blank lines, *space characters* and *tab characters* (i.e., “tabs”) to make programs easier to read.
 - Together, these characters are known as **white space**.
 - White-space characters are normally *ignored* by the compiler.



2.2 First Program in C++: Printing a Line of Text (cont.)

- ▶ `main` is a part of every C++ program.
- ▶ The parentheses after `main` indicate that `main` is a program building block called a **function**.
- ▶ C++ programs typically consist of one or more functions and classes.
- ▶ Exactly *one* function in every program *must* be named `main`.
- ▶ C++ programs begin executing at function `main`, even if `main` is *not* the first function defined in the program.
- ▶ The keyword `int` to the left of `main` indicates that `main` “returns” an integer (whole number) value.
 - A **keyword** is a word in code that is reserved by C++ for a specific use.
 - For now, simply include the keyword `int` to the left of `main` in each of your programs.



2.2 First Program in C++: Printing a Line of Text (cont.)

- ▶ A **left brace**, {, must *begin* the **body** of every function.
- ▶ A corresponding **right brace**, }, must *end* each function's body.
- ▶ A statement instructs the computer to **perform an action**.
- ▶ Together, the quotation marks and the characters between them are called a **string**, a **character string**.
- ▶ We refer to characters between double quotation marks simply as **strings**.
- ▶ Most C++ statements end with a **semicolon** (;), also known as the **statement terminator**.
 - Preprocessing directives (like `#include`) do not end with a semicolon.



2.2 First Program in C++: Printing a Line of Text (cont.)

- ▶ Typically, output and input in C++ are accomplished with **streams** of characters.
- ▶ When a **cout** statement executes, it sends a stream of characters to the **standard output stream object**—**std::cout**—which is normally “connected” to the screen.
- ▶ The **std::** before **cout** is required when we use names that we’ve brought into the program by the preprocessing directive **#include <iostream>**.
 - The notation **std::cout** specifies that we are using a name, in this case **cout**, that belongs to “namespace” **std**.
 - The names **cin** (the standard input stream).



2.2 First Program in C++: Printing a Line of Text (cont.)

- ▶ In the context of an output statement, the << operator is referred to as the **stream insertion operator**.
 - The value to the operator's right, the right **operand**, is inserted in the output stream.
- ▶ The characters \n are *not* printed on the screen.
- ▶ The backslash (\) is called an **escape character**.
 - It indicates that a “special” character is to be output.
- ▶ When a backslash is encountered in a string of characters, the next character is combined with the backslash to form an **escape sequence**.
- ▶ The escape sequence \n means **newline**.
 - Causes the **cursor** to move to the beginning of the next line on the screen.



Escape sequence	Description
\n	Newline. Position the screen cursor to the beginning of the next line.
\t	Horizontal tab. Move the screen cursor to the next tab stop.
\r	Carriage return. Position the screen cursor to the beginning of the current line; do not advance to the next line.
\a	Alert. Sound the system bell.
\\	Backslash. Used to print a backslash character.
\'	Single quote. Used to print a single quote character.
\"	Double quote. Used to print a double quote character.

Fig. 2.2 | Escape sequences.



2.2 First Program in C++: Printing a Line of Text (cont.)

- ▶ When the **return statement** is used at the end of `main` the value `0` indicates that the program has *terminated successfully*.
- ▶ According to the C++ standard, if program execution reaches the end of `main` without encountering a **return** statement, it's assumed that the program terminated successfully—exactly as when the last statement in `main` is a **return** statement with the value `0`.



2.3 Modifying Our First C++ Program

- ▶ Welcome to C++! can be printed several ways.



```
1 // Fig. 2.3: fig02_03.cpp
2 // Printing a line of text with multiple statements.
3 #include <iostream> // allows program to output data to the screen
4
5 // function main begins program execution
6 int main()
7 {
8     std::cout << "Welcome ";
9     std::cout << "to C++!\n";
10 } // end function main
```

Welcome to C++!

Fig. 2.3 | Printing a line of text with multiple statements.



2.3 Modifying Our First C++ Program (cont.)

- ▶ A single statement can print multiple lines by using newline characters.
- ▶ Each time the `\n` (newline) escape sequence is encountered in the output stream, the screen cursor is positioned to the beginning of the next line.
- ▶ To get a blank line in your output, place two newline characters back to back.



```
1 // Fig. 2.4: fig02_04.cpp
2 // Printing multiple lines of text with a single statement.
3 #include <iostream> // allows program to output data to the screen
4
5 // function main begins program execution
6 int main()
7 {
8     std::cout << "Welcome\n\nC++!\n";
9 } // end function main
```

```
Welcome
to
C++!
```

Fig. 2.4 | Printing multiple lines of text with a single statement.



Variables in C++

- ▶ There are several types of Variables in C++, we will demonstrate some of them in the next slides.



INTEGERS

Variables of this type are used to represent Variables of this type are used to represent integers (whole numbers). Declaring a variable to be of type **int** signals to the compiler that it must associate enough memory with the variable's identifier to store an integer value or integer values as the program executes. But there is a (system dependent) limit on the largest and smallest integers that can be stored. Hence C++ also supports the data types **short int** and **long int** which represent, respectively, a smaller and a larger range of integer values than int. Adding the prefix **unsigned** to any of these types means that you wish to represent non-negative integers only.



INTEGERS

Some rules have to be observed when writing integer values in programs:

- ▶ Decimal points cannot be used; although 26 and 26.0 have the same value, "26.0" is not of type "int".
- ▶ Commas cannot be used in integers, so that (for example) 23,897 has to be written as "23897".
- ▶ Integers cannot be written with leading zeros. The compiler will, for example, interpret "011" as an octal (base 8) number, with value 9



REAL NUMBERS

- ▶ Variables of type "float" are used to store real numbers. Plus and minus signs for data of type "float" are treated exactly as with integers, and trailing zeros to the right of the decimal point are ignored. Hence "+523.5", "523.5" and "523.500" all represent the same value. The computer also accepts real numbers in floating-point form (or "scientific notation"). Hence 523.5 could be written as "5.235e+02" (i.e. $5.235 \times 10 \times 10$), and -0.0034 as "-3.4e-03". In addition to "float", C++ supports the types "double" and "long double", which give increasingly precise representation of real numbers, but at the cost of more computer memory.



CHARACTERS

- ▶ Variables of type "char" are used to store character data. In standard C++, data of type "char" can only be a single character (which could be a blank space). These characters come from an available character set which can differ from computer to computer. However, it always includes upper and lower case letters of the alphabet, the digits 0, ... , 9, and some special symbols such as #, £, !, +, -, etc.

STRINGS



- ▶ In C++ a string constant must be enclosed in double quotation marks. Hence we have seen output statements such as:

Std::cout << "is represented as the number";

- ▶ in programs. In fact, "string" is not a fundamental data type such as "int", "float" or "char". Instead, strings are represented as arrays of characters, so we will return to subject of strings later, when we discuss arrays in general.



2.4 Memory Concepts

- ▶ Variable names such as `number1`, `number2` and `sum` actually correspond to **locations** in the computer's memory.
- ▶ Every variable has a name, a type, a size and a value.
- ▶ When a value is placed in a memory location, the value overwrites the previous value in that location; thus, placing a new value into a memory location is said to be **destructive**.
- ▶ When a value is read out of a memory location, the process is **nondestructive**.



Fig. 2.6 | Memory location showing the name and value of variable number1.

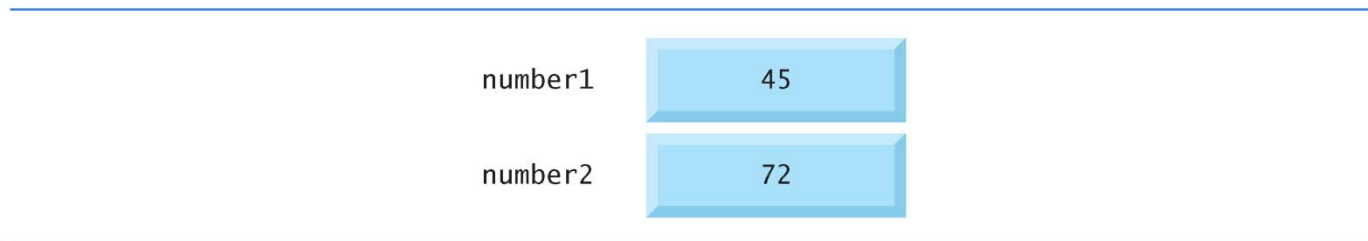
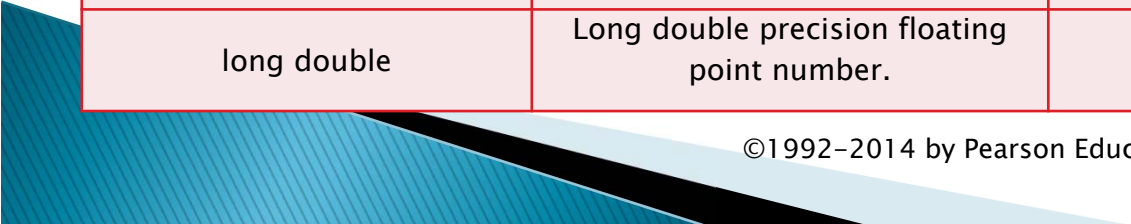


Fig. 2.7 | Memory locations after storing values in the variables for number1 and number2.



number1	45
number2	72
sum	117

Fig. 2.8 | Memory locations after calculating and storing the sum of number1 and number2.



Type	Description	Typical Size in Bits	Minimal Range
char	Character or small integer.	8 (1 byte)	−127 to 127
unsigned char	Character or small integer.	8 (1 byte)	0 to 255
signed char	Character or small integer.	8 (1 byte)	−127 to 127
int	Integer.	32 (4 byte)	−32,767 to 32,767
unsigned int	Integer.	32 (4 byte)	0 to 65,535
signed int	Integer.	32 (4 byte)	same as int
short int	Short Integer.	16 (2 byte)	−32,767 to 32,767
unsigned short int	Short Integer.	16 (2 byte)	0 to 65,535
signed short int	Short Integer.	16 (2 byte)	same as short int
long int	Long integer.	32 (4 byte)	−2,147,483,647 to 2,147,483,647
signed long int	Long integer.	32 (4 byte)	same as long int
unsigned long int	Long integer.	32 (4 byte)	0 to 4,294,967,295
float	Floating point number.	32 (4 byte)	+/- 3.4e +/- 38 (~7 digits)
double	Double precision floating point number.	64(8 byte)	+/- 1.7e +/- 308 (~15 digits)
long double	Long double precision floating point number.	64(8 byte)	+/- 1.7e +/- 308 (~15 digits)



2.5 Another C++ Program: Adding Integers

- ▶ The next program obtains two integers typed by a user at the keyboard, computes the sum of these values and outputs the result using `std::cout`.
- ▶ Figure 2.5 shows the program and sample inputs and outputs.



```
1 // Fig. 2.5: fig02_05.cpp
2 // Addition program that displays the sum of two integers.
3 #include <iostream> // allows program to perform input and output
4
5 // function main begins program execution
6 int main()
7 {
8     // variable declarations
9     int number1 = 0; // first integer to add (initialized to 0)
10    int number2 = 0; // second integer to add (initialized to 0)
11    int sum = 0; // sum of number1 and number2 (initialized to 0)
12
13    std::cout << "Enter first integer: "; // prompt user for data
14    std::cin >> number1; // read first integer from user into number1
15
16    std::cout << "Enter second integer: "; // prompt user for data
17    std::cin >> number2; // read second integer from user into number2
18
19    sum = number1 + number2; // add the numbers; store result in sum
20
21    std::cout << "Sum is " << sum << std::endl; // display sum; end line
22 }
```

Fig. 2.5 | Addition program that displays the sum of two integers.



```
Enter first integer: 45  
Enter second integer: 72  
Sum is 117
```

Fig. 2.5 | Addition program that displays the sum of two integers.



2.5 Another C++ Program: Adding Integers (cont.)

- ▶ **Declarations** introduce identifiers into programs.
- ▶ The identifiers `number1`, `number2` and `sum` are the names of **variables**.
- ▶ A variable is a location in the computer's memory where a value can be stored for use by a program.
- ▶ Variables `number1`, `number2` and `sum` are data of type **int**, meaning that these variables will hold **integer** values, i.e., whole numbers such as 7, -11, 0 and 31914.
- ▶ All variables *must* be declared with a *name* and a *data type* before they can be used in a program.
- ▶ If more than one name is declared in a declaration (as shown here), the names are separated by commas (,); this is referred to as a **comma-separated list**.



Error-Prevention Tip 2.1

Although it's not always necessary to initialize every variable explicitly, doing so will help you avoid many kinds of problems.



Good Programming Practice 2.4

Declare only one variable in each declaration and provide a comment that explains the variable's purpose in the program.



2.5 Another C++ Program: Adding Integers (cont.)

- ▶ Data type `double` is for specifying real numbers, and data type `char` for specifying *character data*.
- ▶ Real numbers are numbers with decimal points, such as 3.4, 0.0 and -11.19.
- ▶ A `char` variable may hold only a single lowercase letter, a single uppercase letter, a single digit or a single special character (e.g., \$ or *).
- ▶ Types such as `int`, `double` and `char` are called *fundamental types*.
- ▶ Fundamental-type names are keywords and therefore *must* appear in all lowercase letters.



2.5 Another C++ Program: Adding Integers (cont.)

- ▶ A variable name is any valid **identifier** that is *not* a keyword.
- ▶ An identifier is a series of characters consisting of letters, digits and underscores (`_`) that does not begin with a digit.
- ▶ C++ is **case sensitive**—uppercase and lowercase letters are different, so **a1** and **A1** are *different* identifiers.



Example:

Correct

Count

test23

high_balance

Cin

Incorrect

1 Count

hi!there

high..balance

cin

- ▶ C++ places no limit on the length of the name; therefore, all characters in a name are significant. But some OS impose restriction on the length of variable name.



► Some Keywords

struct	int	double	auto
switch	long	else	break
typedef	register	enum	case
union	return	extern	char
unsigned	short	float	const
void	signed	for	continue
volatile	sizeof	goto	default
while	static	if	do



2.5 Another C++ Program: Adding Integers (cont.)

- ▶ Declarations of variables can be placed almost anywhere in a program, but they must appear *before* their corresponding variables are used in the program.



2.5 Another C++ Program: Adding Integers (cont.)

- ▶ A **prompt** it directs the user to take a specific action.
- ▶ A **cin** statement uses the **input stream object cin** (of namespace **std**) and the **stream extraction operator, >>**, to obtain a value from the keyboard.
- ▶ Using the stream extraction operator with **std::cin** takes character input from the standard input stream, which is usually the keyboard.



2.5 Another C++ Program: Adding Integers (cont.)

- ▶ When the computer executes an input statement that places a value in an `int` variable, it waits for the user to enter a value for variable `number1`.
- ▶ The user responds by typing the number (as characters) then pressing the *Enter* key (sometimes called the Return key) to send the characters to the computer.
- ▶ The computer converts the character representation of the number to an integer and assigns (i.e., copies) this number (or *value*) to the variable `number1`.
- ▶ Any subsequent references to `number1` in this program will use this same value.



2.5 Another C++ Program: Adding Integers (cont.)

- ▶ In this program, an assignment statement adds the values of variables `number1` and `number2` and assigns the result to variable `sum` using the **assignment operator =**.
 - Most calculations are performed in assignment statements.
- ▶ The `=` operator and the `+` operator are called **binary operators** because each has two operands.



2.5 Another C++ Program: Adding Integers (cont.)

- ▶ `std::endl` is a so-called **stream manipulator**.
- ▶ The name `endl` is an abbreviation for “end line” and belongs to namespace `std`.
- ▶ The `std::endl` stream manipulator outputs a newline, then “flushes the output buffer.”
 - This simply means that, on some systems where outputs accumulate in the machine until there are enough to “make it worthwhile” to display them on the screen, `std::endl` forces any accumulated outputs to be displayed at that moment.
 - This can be important when the outputs are prompting the user for an action, such as entering data.



2.5 Another C++ Program: Adding Integers (cont.)

- ▶ Using multiple stream insertion operators (<<) in a single statement is referred to as **concatenating**, **chaining** or **cascading stream insertion operations**.
- ▶ Calculations can also be performed in output statements.